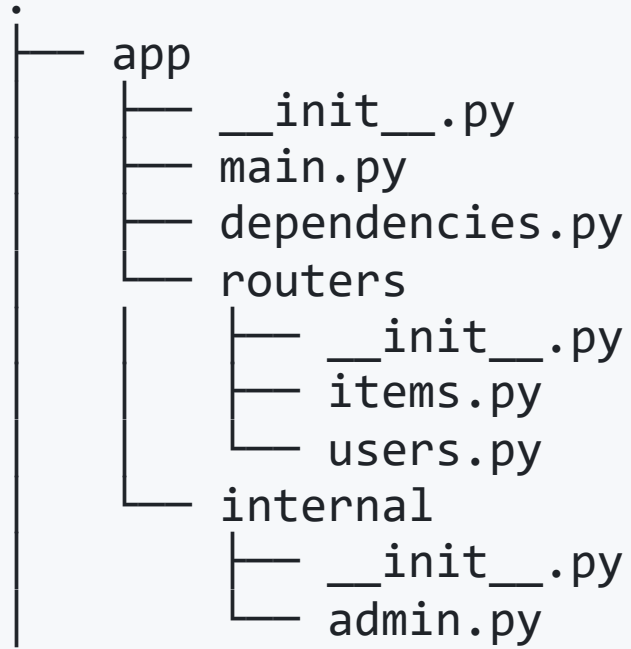


Large Application

- 애플리케이션이나 웹 API를 만들 때, 모든 것을 하나의 파일에 작성하는 경우는 드물다.
- FastAPI는 유연성을 유지하면서도 애플리케이션을 구조화할 수 있게 해주는 편리한 도구를 제공한다.

예시 파일 구조



- `app/` 디렉터리는 최상위 폴더
 - 빈 파일 `app/__init__.py` 에 의해 "Python package" ("Python modules"의 모음)인 `app` 이 된다.
 - `app/main.py` : module이고, `app.main` 으로 접근
- `app/routers/` : "Python subpackage", `app.routers`
- `app/routers/items.py` : submodule, `app.routers.items`

Package app
app/__init__.py

Module app.main
app/main.py

Module app.dependencies
app/dependencies.py

Subpackage app.routers
app/routers/__init__.py

Module app.routers.items
app/routers/items.py

Module app.routers.users
app/routers/users.py

Subpackage app.internal
app/internal/__init__.py

Module app.internal.admin
app/internal/admin.py

파일 구조

```

.
├── app
│   ├── __init__.py
│   ├── main.py
│   ├── dependencies.py
│   └── routers
│       ├── __init__.py
│       ├── items.py
│       └── users.py
└── internal
    ├── __init__.py
    └── admin.py

# 'app'은 Python 패키지입니다
# 이 파일로 'app'이 'Python 패키지'가 됩니다
# 'main' 모듈, 예: import app.main
# 'dependencies' 모듈, 예: import app.dependencies
# 'routers'는 'Python 하위 패키지'입니다
# 이 파일로 'routers'가 'Python 하위 패키지'가 됩니다
# 'items' 서브모듈, 예: import app.routers.items
# 'users' 서브모듈, 예: import app.routers.users
# 'internal'은 'Python 하위 패키지'입니다
# 이 파일로 'internal'이 'Python 하위 패키지'가 됩니다
# 'admin' 서브모듈, 예: import app.internal.admin
```

APIRouter

- `APIRouter` 는 규모가 큰 애플리케이션 개발 시, ****API 엔드포인트(라우트)****를 기능이나 도메인 별로 분리하고 체계적으로 구조화하는 *M/L/ FastAPI* 인스턴스
 - 모든 API 라우트를 하나의 `main.py` 파일에 작성하면 유지보수가 힘들다

💡 쉽게 이해하는 비유

- `FastAPI()` (메인 앱) ➡ 본사 오피스 (전체 서비스를 총괄하고 실행함)
- `APIRouter()` (라우터) ➡ 전문 부서 / 지사 (인사부, 회계부, 개발부처럼 특정 업무만 모아서 처리함)

```
from fastapi import APIRouter

router = APIRouter()

@router.get("/users/", tags=["users"])
async def read_users():
    return [{"username": "Rick"}, {"username": "Morty"}]

@router.get("/users/me", tags=["users"])
async def read_user_me():
    return {"username": "fakecurrentuser"}

@router.get("/users/{username}", tags=["users"])
async def read_user(username: str):
    return {"username": username}
```